United International University
Department of Computer Science and Engineering

Agri Rover

A Cable-Suspended Crop Monitoring and AI-Based Diagnostic System

Final Project Report

Fokir Mostakim Billa Sobuj — 0112430197
Shaikh Jubair — 0112420017
Tahmidur Rahman Foysal — 0112430199
Najif Jawoad Jim — 0112330545
Md. Rubyat Simum Mahi — 0112330575
Jakaria Shikder — 0112230715
October 2026

Abstract

Manual crop inspection is slow, varies between inspectors and often finds diseases, pests and weeds only after they have spread, while wheeled field robots can sink in mud and damage crops. This report presents *Agri Rover*, a cable-suspended rover that monitors crops from above while travelling along a rope-and-pulley ropeway, so it never touches the soil or the plants. The rover moves, stops at scanning points and reverses when an IR sensor detects a pole at the end of the rope. An ESP32-CAM acts as the core controller: it reads a DHT11 temperature and humidity sensor, captures live video, drives the DC motors through an L298N motor driver and streams video and sensor data over Wi-Fi to a laptop. A Python backend running YOLOv8 marks diseases, pests and weeds, and a web dashboard shows live video, readings, annotated images and a detection history. Quantitative performance measurements were not part of the available project documentation and are not reported.

Keywords: Agri Rover, Smart Agriculture, ESP32-CAM, YOLOv8, DHT11, IR Sensor, IoT, Autonomous Rover, Crop Monitoring

I. Introduction

Crop production depends on early recognition of infections, insect infestations and competing weeds, together with continuous awareness of growing conditions. Sensors, connectivity and data analysis are

increasingly used for this under the heading of smart or precision agriculture [1, 2]. Continuous monitoring of an open field is nevertheless difficult: fields are large, conditions change hourly, and observations that rely on a person walking the rows are sparse in time and depend on the individual inspector.

A. Problem Statement

The project documentation identifies four related problems: (1) manual checks are slow and inconsistent; (2) infrequent inspection lets diseases, pests and weeds spread before they are noticed; (3) wheeled ground robots may sink in mud and trample the crop they are meant to protect; and (4) farmers lack continuous readings of field conditions between inspections.

B. Motivation and Objectives

Agri Rover hangs from a rope and pulley ropeway and moves above the crops, so it has no wheel contact with soil or plants and can repeat the same path without altering the field. A camera and an environmental sensor on the moving rover, linked to a processing system and dashboard, address the lack of continuous data. The objectives are:

- Develop a cable-suspended agricultural rover that moves autonomously along the rope.
- Monitor temperature and humidity with a DHT11 and detect the pole or rope end with an IR sensor so that the rover reverses automatically.
- Capture live crop video with the ESP32-CAM and detect diseases, pests and weeds with YOLOv8.
- Display live sensor, video and diagnostic information on a web dashboard, reducing the need for manual inspection.

II. Literature Review and Innovation

Smart farming and IoT. Wolfert *et al.* reviewed big data in smart farming [1], and Ray surveyed IoT applications in agriculture, including sensor networks, wireless links and deployment challenges such as cost and energy [2]. Agri Rover follows this pattern, but its sensing node is a mobile platform carrying its sensors along a fixed path rather than a set of fixed nodes.

Agricultural robots and cable-suspended platforms. Reviews of agricultural robots stress how hard reliable outdoor operation is in unstructured environments [3]. The ETH field phenotyping platform is a cable-suspended multi-sensor system covering about one hectare at 2–5 m above the canopy that, unlike ground vehicles, imposes no traffic on the field [4]; the Field Scanalyzer positions a sensor array above the crop on fixed rails [5]. These are large research installations with high-end payloads, whereas Agri Rover uses a single rope, an ESP32-CAM and a DHT11. The shared principle is that sensing from above avoids ground traffic and soil conditions.

Computer vision and YOLO. Deep learning has been surveyed for plant disease and weed identification [6], and convolutional networks can classify crop diseases from leaf images [7], although results on curated image sets do not automatically carry over to a rover camera outdoors. YOLO detectors work in a single pass and are designed for real-time use [8, 9]; YOLOv8 is distributed by Ultralytics [10].

Innovation. The contribution lies in the integration rather than any single element: overhead cable-suspended movement above muddy or planted ground; autonomous stop-scan-reverse operation; visual observation paired with local temperature and humidity at each stop; YOLOv8 diagnostics; and one dashboard combining all of it in a low-cost prototype. The report makes no claim that the approach is the first of its kind.

III. Project Overview and System Architecture

Agri Rover is carried on a rope and driven along it by DC motors acting on a pulley. The DHT11, the IR sensor and the camera feed the ESP32-CAM, which is the central controller. Two cooperating paths result (Fig. 1). In the **control and movement path**, the ESP32-CAM sends control signals to the L298N driver, which supplies power and direction to the DC motors and pulley; the IR sensor tells the controller when a pole or rope end is reached. In the **data and diagnostic path**, the ESP32-CAM sends video and sensor data over Wi-Fi to a laptop, where a Python backend passes the video to YOLOv8 and the results appear on the web dashboard together with the sensor data. The laptop does the heavy computation, so the rover stays a compact embedded unit.

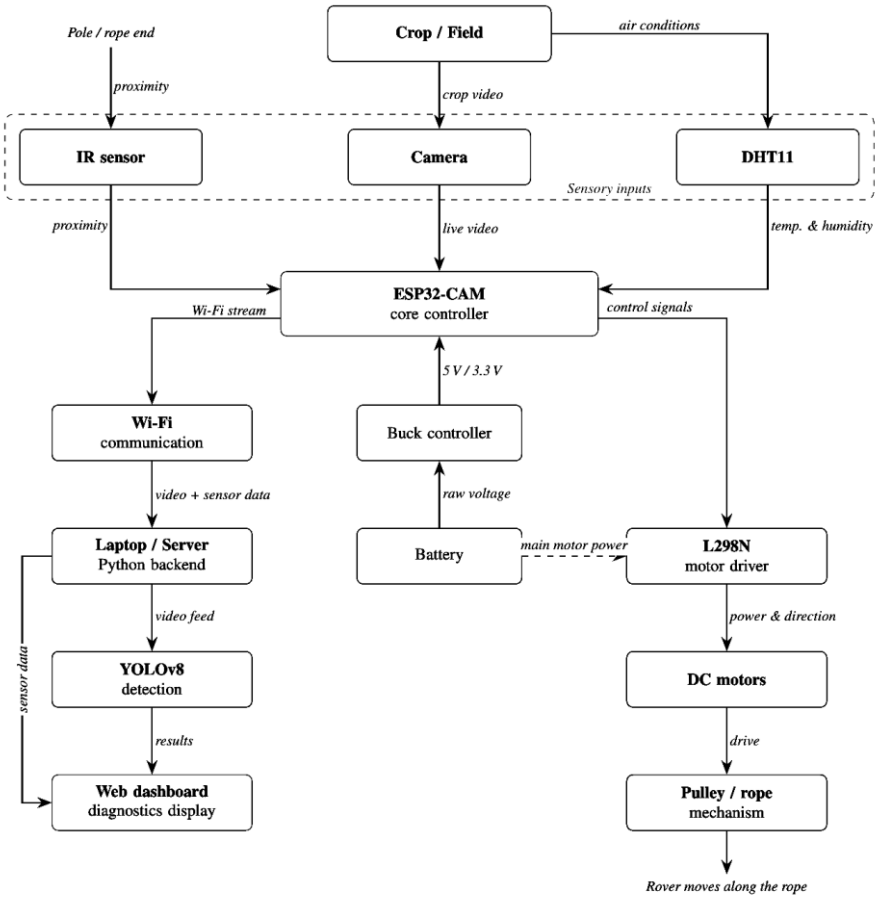


Fig. 1. Overall system architecture of Agri Rover, redrawn from the project presentation data-flow map (battery and buck controller shown for completeness).

Information flows one way for each purpose: sensors → ESP32-CAM; ESP32-CAM → motor driver and motors; ESP32-CAM → Wi-Fi → laptop → YOLOv8 → dashboard. The documentation describes no path by which the laptop or dashboard commands the rover, so none is assumed.

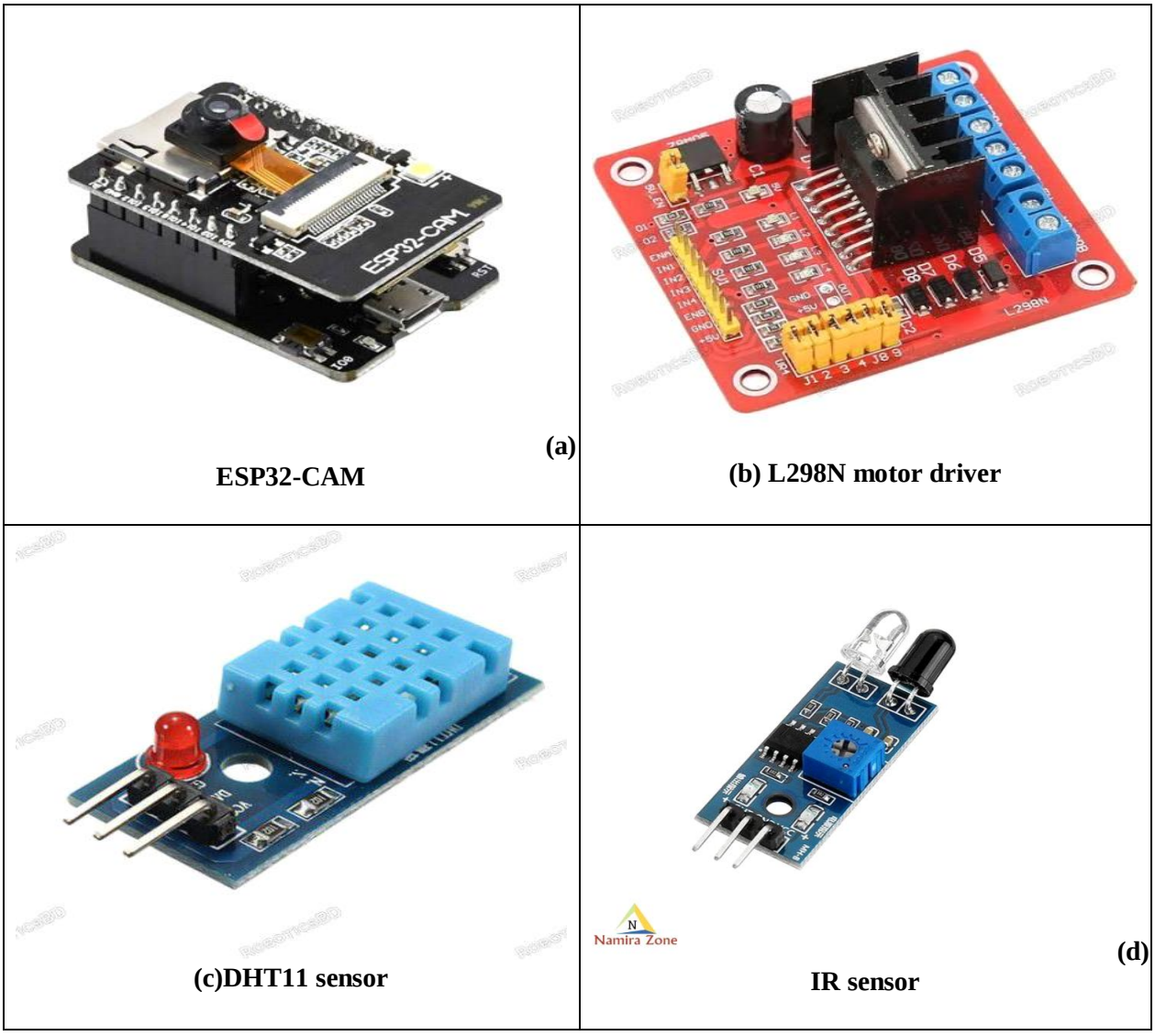
IV. Components

Table 1 lists the documented components; Figs. 2 and 3 show them and the text below summarises each role. Quantities, ratings and part numbers were not given in the project documentation.

Table 1. Component list

Component	**Role in Agri Rover**
ESP32-CAM (with camera)	Core controller; reads sensors, captures live video, issues motor signals, streams over Wi-Fi
L298N motor driver	Controls speed and direction of the DC motors
DC motors + pulley/rope	Drive the rover along the rope above the crop
DHT11	Temperature and humidity at each stop
IR sensor	Detects poles / rope end so the rover reverses
Battery *and buck controller*	Power source; supplies 5 V/3.3 V to the ESP32-CAM (shown in the architecture diagram only)
Laptop / server	Receives the stream; runs the Python backend and YOLOv8

**Shown in the architecture diagram but not in the presentation component table.*



(a) ESP32-CAM

(b) L298N motor driver

(c)DHT11 sensor

(d) IR sensor

Fig. 2. Major electronic components used in Agri Rover: (a) ESP32-CAM, (b) L298N motor driver, (c) DHT11 sensor and (d) IR sensor.

ESP32-CAM

Function: Core controller of the rover; it combines a Wi-Fi microcontroller [11] with a camera interface.

How it works in the project: It reads the DHT11 and IR sensor, captures video, signals the L298N and streams video and sensor data to the laptop.

L298N motor driver

Function: Interface between the low-power controller and the motors; the L298 is a dual full-bridge driver for inductive loads [12].

How it works in the project: It takes control signals from the ESP32-CAM and applies speed and direction to the DC motors from the main motor supply.

DHT11 temperature and humidity sensor

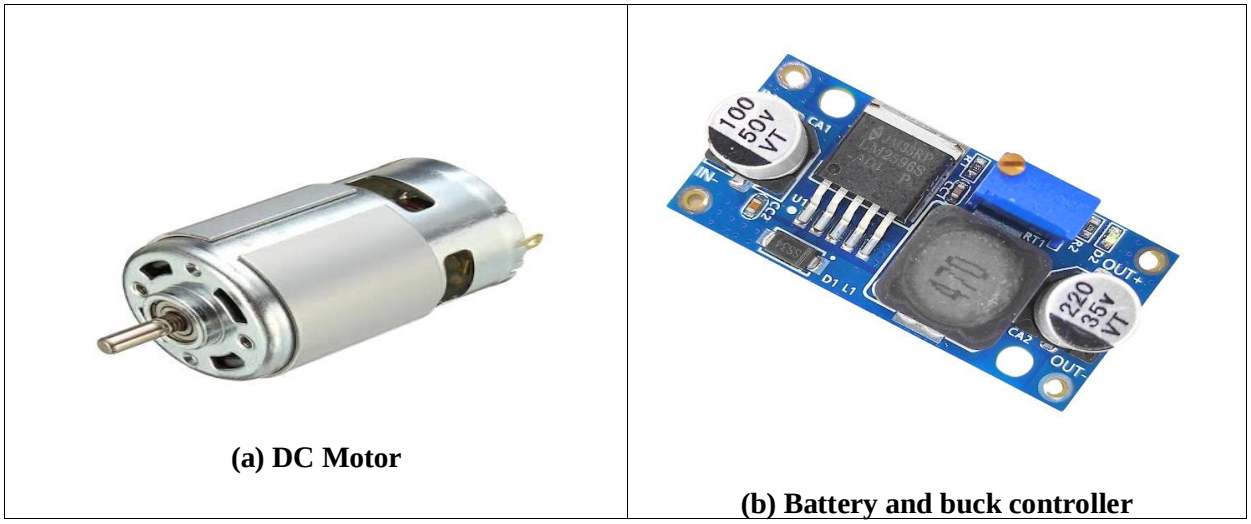
Function: Low-cost digital sensor for temperature and relative humidity [13].

How it works in the project: It is read at each scanning stop so the dashboard shows conditions at the rover's current position.

IR sensor

Function: Provides proximity information used to detect poles and the rope end.

How it works in the project: When it detects the pole, the ESP32-CAM reverses the motor direction. Model, mounting position and range are not specified.



(a) DC Motor

(b) Battery and buck controller

Fig. 3. Mechanical and power parts: (a) DC motor and (b) battery and buck controller.

DC motors and pulley/rope mechanism

Function: Provide the mechanical power that moves the rover along the rope.

How it works in the project: The L298N drives the motors; reversing polarity reverses travel, which is how the rover runs back after the IR sensor detects the pole. Motor ratings and rope length are not specified.

Battery, buck controller and laptop

Function: The battery powers the motors directly and, through the buck controller, the ESP32-CAM; the laptop is the server for the Python backend and YOLOv8.

How it works in the project: Wi-Fi carries video and sensor data to the laptop, which removes the cable between rover and computer but limits operation to the Wi-Fi range.

V. Working Principle and Control

Operation is a repeating stop-and-scan cycle (Fig. 4, Fig. 5): (1) the rover moves along the rope; (2) it stops at the scanning point; (3) the camera captures video and the DHT11 records temperature and humidity; (4) the ESP32-CAM sends the data over Wi-Fi; (5) the Python backend processes the stream with YOLOv8; (6) diseases, pests and weeds are identified and marked; (7) results appear on the dashboard; (8) when the IR sensor detects the pole the rover reverses; (9) it travels back along the rope; (10) the cycle repeats. The presentation indicates 2 s of movement and 5 s for stopping, scanning and reading the DHT11; how these are configured in the firmware is not described.

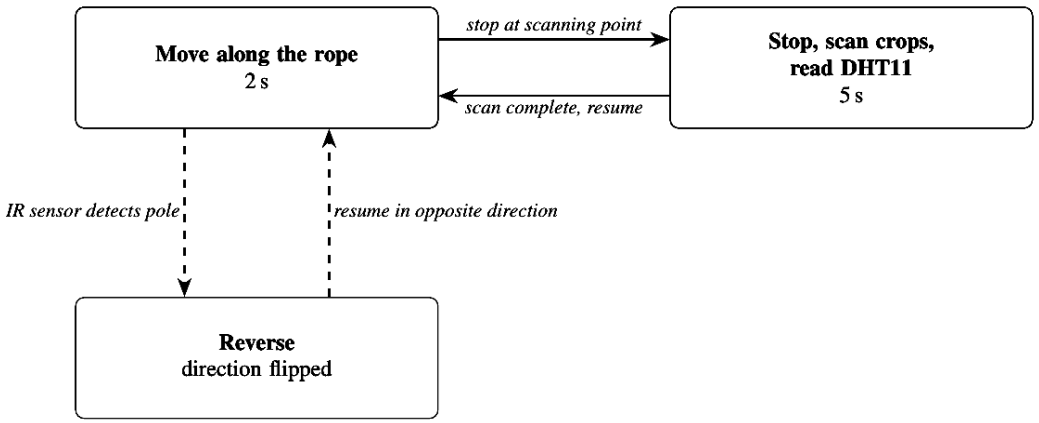


Fig. 4. Stop-and-scan operational cycle. Durations are those shown on the project presentation.

The control logic is the documented sequence *Move → Stop → Scan → Read sensor → Process data → Continue → Detect pole → Reverse → Repeat*, which runs back and forth along the whole rope without an operator. No closed-loop speed control, position feedback, obstacle avoidance beyond the poles or scheduling is documented, so none is assumed; the IR threshold and firmware details are not specified.

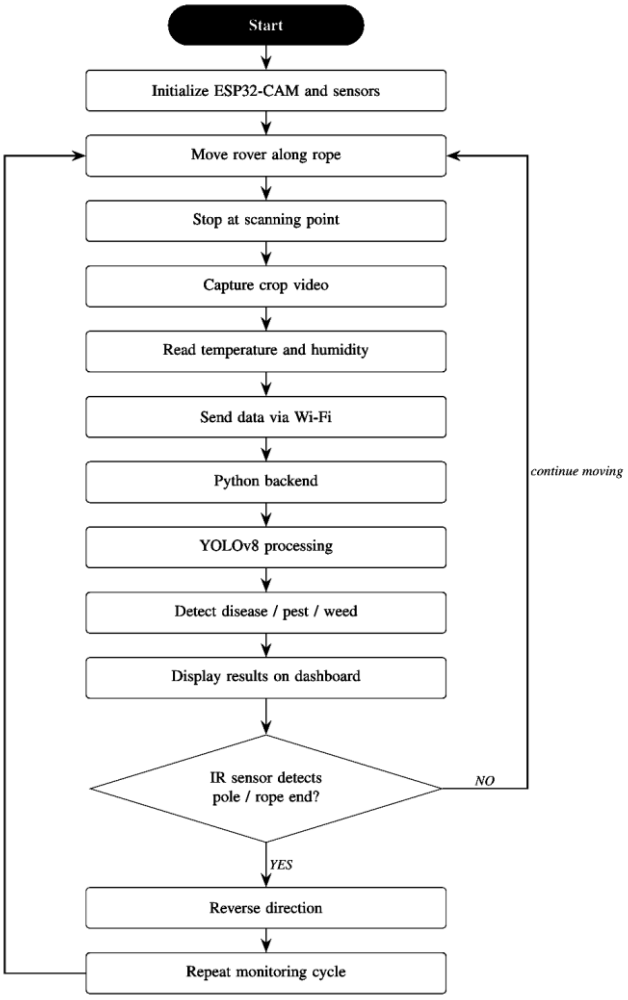


Fig. 5. System flowchart of the Agri Rover monitoring cycle.

VI. AI-Based Crop Diagnostics (YOLOv8)

The ESP32-CAM video is sent over Wi-Fi to the Python backend, YOLOv8 processes each frame in a single pass and marks detections with bounding boxes and labels [8, 9], and the annotated images go to the dashboard (Fig. 6). Table 2 gives the three documented detection categories.

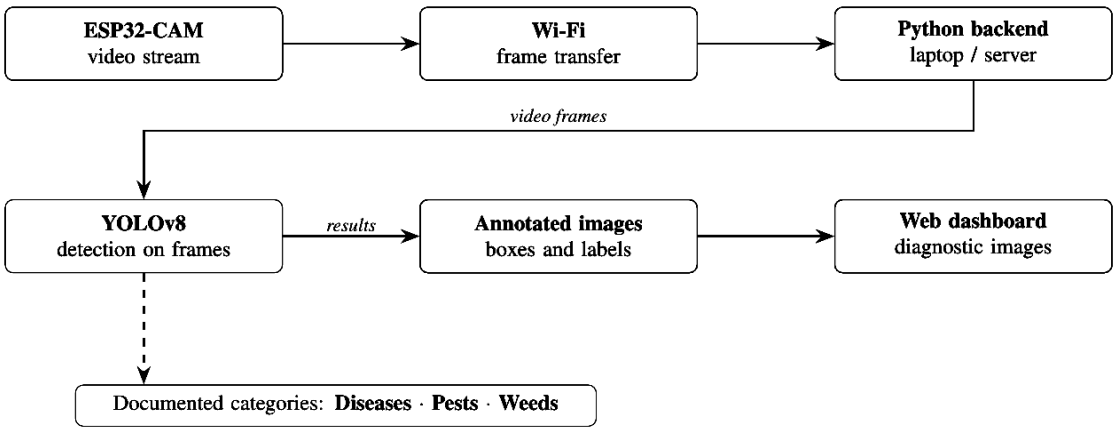


Fig. 6. YOLOv8 diagnostic process from the ESP32-CAM stream to annotated dashboard images. No real detection output was included in the project materials.

Table 2. Documented detection categories

Category	Description in the project documentation
Diseases	Leaf and plant infections
Pests	Insects on leaves and stems
Weeds	Unwanted plants between rows

The YOLOv8 variant, training dataset, number of classes, training procedure, confidence threshold, accuracy and frame rate are *not specified* in the project documentation and no detection performance is claimed.

VII. Real-Time Dashboard

The web dashboard (Fig. 7) combines four documented views: **live video** of what the rover sees; **temperature and humidity** from the DHT11 at each stop; **AI diagnostic images** with boxes and labels; and a **detection history** for tracking crop health over repeated passes. Together they turn raw data into usable monitoring information: annotation directs attention, readings give context, and history adds the time dimension. The web technologies and storage format are not specified.

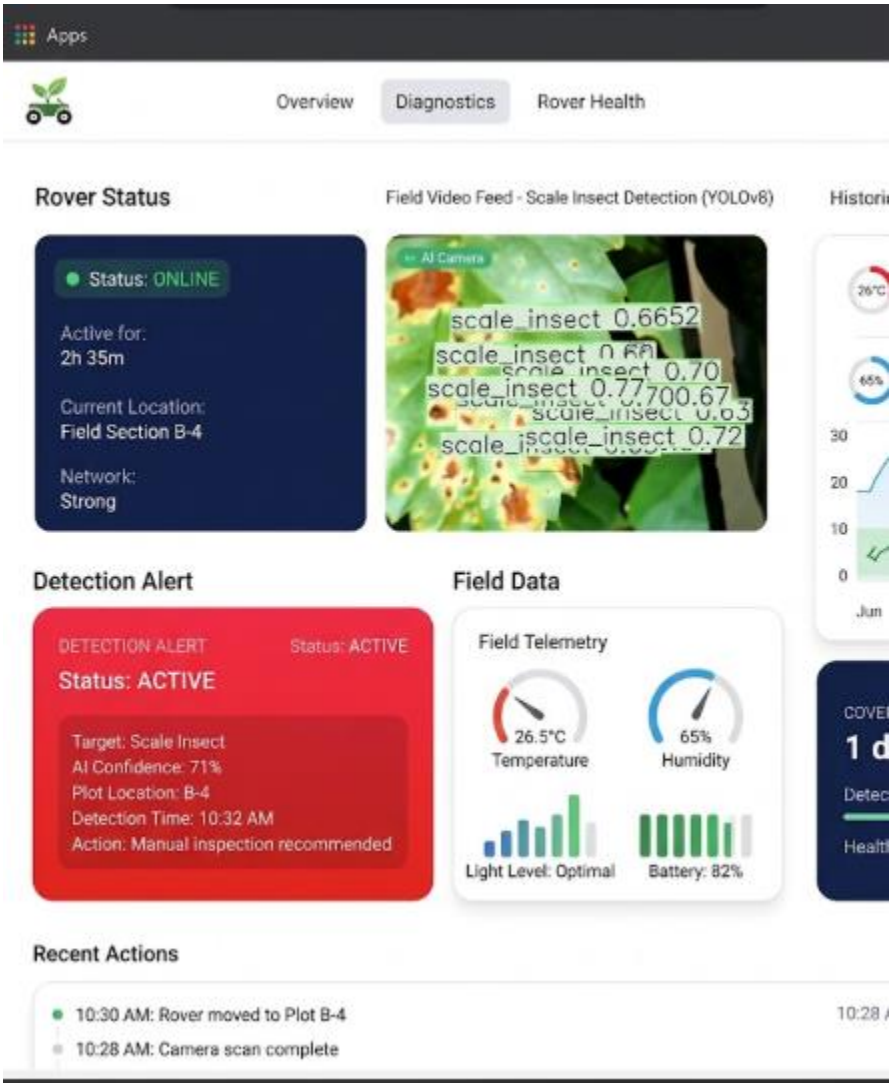


Fig. 7. Schematic of the dashboard showing the four documented panels (a schematic drawn for this report, not a screenshot).

VIII. Prototype and Results

The prototype is a rope-supported structure along which the rover travels, moved by DC motors on a pulley through the L298N and carrying the DHT11, IR sensor and ESP32-CAM. It is powered from a battery through a buck controller. The project presentation contains no photograph of the prototype and does not specify rope length, rope height, sensor mounting positions or the number of poles; Fig. 8 is reserved for the team's photograph.

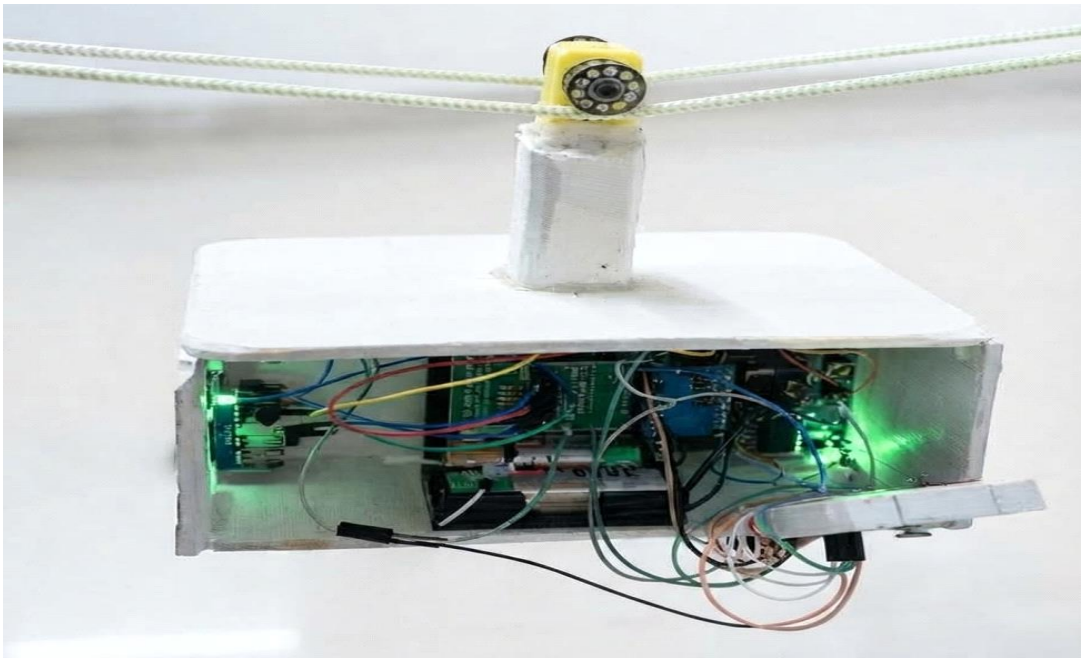


Fig. 8. Physical Agri Rover prototype (photograph to be inserted by the team).

The materials describe how the system works but contain no test logs, so the observations are descriptive: the rover moves along the rope on the pulley; it repeats a 2 s move / 5 s scan cycle; the IR sensor flips its direction at the pole; the DHT11 readings and YOLOv8-annotated video are shown on the dashboard. Measured detection accuracy, speed, battery duration, Wi-Fi range, latency and sensor error were not provided and none is reported; they are recommended as the next evaluation step. A cost table could not be completed because component prices and quantities were not documented.

IX. Advantages, Limitations and Future Scope

Advantages. No crop damage from ground contact; operation above muddy ground; a low-cost concept based on small embedded parts (not yet quantified); early-detection support by marking diseases, pests and weeds; and continuous, repeated monitoring with a live dashboard and history that reduces manual inspection.

Table 3. Limitations

Limitation	**Effect**
Limited to the rope path	Only the crop beneath the rope can be observed
Wind and weather	May disturb the suspended rover and measurement quality; limits are not documented
Wi-Fi range	Limits the distance over which video and data reach the laptop
DHT11 accuracy	Modest accuracy [13]; suitable for general conditions only
No reported YOLOv8 accuracy	Reliability of the diagnoses cannot yet be judged

Future scope (none implemented): solar power for longer operation; precision spraying directed by detections; mobile alerts to farmers; and cloud analytics for long-term crop-health trends.

X. Conclusion

Agri Rover combines autonomous movement along a rope, DHT11 sensing, ESP32-CAM live video, YOLOv8 diagnostics, Wi-Fi communication and a real-time dashboard. By hanging above the crop it

avoids ground contact, plant damage and muddy terrain, and its stop-and-scan cycle supports continuous monitoring. It is confined to the rope path, depends on weather and Wi-Fi, and uses a low-accuracy sensor; because no measured performance is available, the report does not claim improved yield or solved monitoring. Measuring accuracy, range and endurance, documenting cost and adding the proposed extensions are the natural next steps.

XI. Team Contribution

The project was carried out by a team of six members. Fokir Mostakim Billa Sobuj and Shaikh Jubair made the largest contributions, and the remaining four members contributed equally, as shown in Table 4.

Table 4. Team contribution

Team member	Student ID	Contribution
Fokir Mostakim Billa Sobuj	0112430197	Managed the entire project, designed overall system architecture, completed full hardware assembly, circuit design, sensor integration, L298N motor setup, and core C++ (Arduino) programming
Shaikh Jubair	0112420017	AI & Software Backend: Developed the Python backend server, integrated YOLOv8 AI models, and designed the live web dashboard.
Tahmidur Rahman Foysal	0112430199	Project Management & QA: Component sourcing, dashboard UI testing, and overall quality assurance monitoring.
Najif Jawoad Jim	0112330545	Field Testing & Trial: Field-level trial execution, ropeway mechanism testing, and environmental data collection.
Md. Rubyat Simum Mahi	0112330575	Research & Documentation: Project proposal creation, literature review, requirement analysis, and final report writing.
Jakaria Shikder	0112230715	Documentation Assistant: Assisted in organizing background research, structuring report sections, and formatting academic documents.